

UNIVERSAL AUTOMATED NEWS SLATE

MASTER FRAMEWORK BLUEPRINT

A Replicable, Zero-Backend, Fully Automated White-Label Platform for Niche Intelligence Portals
Verticals: Finance | AI & Tech | Fitness & Health | Real Estate | Medical

1. EXECUTIVE ARCHITECTURE & ZERO-QUOTA PATTERN

The Universal Automated News Slate Framework delivers a real-time, AI-curated intelligence portal with ZERO server infrastructure fees and 0 Firestore database read costs for high-frequency widgets.

Core Architectural Pillars:

- Static CDN First (0 Firestore Read Quota): Live tickers & heatmaps fetch from public/niche-data.json.
- Gemini AI Autonomous Curation: Google Gemini API generates 3 key bullets & impact tags.
- GitHub Actions Automated Pipeline: Headless cron runs twice daily (9 AM & 5 PM IST).
- Multi-Client White-Label Architecture: Independent Firebase Projects & GitHub Repos per customer.

2. DOMAIN ADAPTATION MATRIX

Vertical	Categories	Key Widgets	Data Sources
AI & Tech	LLMs, Robotics, AI Ethics, Dev Tools	Model Benchmark Heatmap	ArXiv, TechCrunch
Fitness	Hypertrophy, Nutrition, Biohacking	Macro Estimator, Calorie Chart	PubMed, Men's Health
Finance	Equities, Mutual Funds, PMS/AIF	5-Day FII/DII Chart, Sector Heatmap	Economic Times, GF
Real Estate	Commercial, REITs, PropTech	Rental Yield Tracker, Circle Rates	Housing.com, PropTiger

3. STEP-BY-STEP REPLICATION & AI SCRAPER PIPELINE

Below is the complete, copy-paste ready AI Curation Engine script (scripts/curate-news.cjs):

```
const fs = require('fs');
const { GoogleGenerativeAI } = require('@google/generative-ai');
const admin = require('firebase-admin');

// Initialize Firebase Admin
const serviceAccount = JSON.parse(process.env.FIREBASE_SERVICE_ACCOUNT_KEY);
admin.initializeApp({ credential: admin.credential.cert(serviceAccount) });
const db = admin.firestore();
const genAI = new GoogleGenerativeAI(process.env.GEMINI_API_KEY);

async function curateNicheNews() {
  console.log('ðŸš€ Starting AI Curation Engine...');
  const rawArticles = await fetchRawArticles();
  const model = genAI.getGenerativeModel({ model: 'gemini-1.5-flash' });

  for (const item of rawArticles) {
    const prompt = `Summarize in 3 bullet points, assign category & impact (High/Medium/Standard):
    Title: ${item.title} Content: ${item.content}`;

    const result = await model.generateContent(prompt);
    const parsed = JSON.parse(result.response.text().replace(/``json|``/g, '').trim());

    const docId = Buffer.from(item.title).toString('hex').slice(0, 20);
    await db.collection('articles').doc(docId).set({
      title: item.title,
      summary: parsed.summary,
      category: parsed.category,
      impactLevel: parsed.impactLevel,
      tags: parsed.tags,
      source: item.source,
      sourceUrl: item.link,
      curatedDate: new Date().toISOString().split('T')[0],
      createdAt: admin.firestore.FieldValue.serverTimestamp(),
    }, { merge: true });
  }
}

curateNicheNews();
```

4. LIVE DATA SCRAPER (STATIC CDN FEED)

```
// scripts/fetch-niche-data.cjs - Generates public/niche-data.json (0 Firestore Reads)
const fs = require('fs');
const path = require('path');

async function fetchLiveNicheMetrics() {
  const tickerData = await fetchLiveTickerFromWeb();
  const sectorData = await fetchLiveSectorHeatmapFromWeb();

  const payload = { ticker: tickerData, sectors: sectorData, updatedAt: new Date().toISOString() };
  fs.writeFileSync(path.join(__dirname, '../public/niche-data.json'), JSON.stringify(payload, null, 2));
}

fetchLiveNicheMetrics();
```

5. GITHUB ACTIONS AUTOMATED DEPLOYMENT WORKFLOW

File location: .github/workflows/curate.yml

```
name: ⌚ Daily AI Curation & Deployment Pipeline

on:
  schedule:
    - cron: '33 3 * * *' # 9:03 AM IST
    - cron: '3 12 * * *' # 5:33 PM IST
  workflow_dispatch:

jobs:
  curate-and-deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - uses: actions/setup-node@v3
        with: { node-version: 18 }
      - run: npm ci
      - run: node scripts/fetch-niche-data.cjs
      - run: node scripts/curate-news.cjs
      env:
        GEMINI_API_KEY: ${ secrets.GEMINI_API_KEY }
        FIREBASE_SERVICE_ACCOUNT_KEY: ${ secrets.FIREBASE_SERVICE_ACCOUNT_KEY }
      - run: npm run build
      - uses: FirebaseExtended/action-hosting-deploy@v0
        with:
          repoToken: '${ secrets.GITHUB_TOKEN }'
          firebaseServiceAccount: '${ secrets.FIREBASE_SERVICE_ACCOUNT }'
          channelId: live
          projectId: YOUR_NEW_FIREBASE_PROJECT_ID
```

6. 30-MINUTE CLIENT ONBOARDING CHECKLIST

1. Create New Firebase Project: Enable Cloud Firestore & Hosting on Console.
2. Create GitHub Repo: Push cloned template code to client's GitHub repository.
3. Add Secrets to GitHub: Set GEMINI_API_KEY & FIREBASE_SERVICE_ACCOUNT.
4. Customize Branding: Update logo.jpeg, Header titles, and advisory contact details.
5. Trigger Initial Deployment: Run `npm run build` and `firebase deploy --only hosting`.

